# AI Study Assistant: Project Proposal

**Ali Belhrak · CIS 641 System Analysis and Design**

# 1. Introduction

**I propose the AI Study Assistant, a web application that puts a student's courses, assignments, exams, deadlines, study schedule and course materials in one place. Its AI component reads the student's upcoming deadlines and recorded study time, then recommends which course to focus on and adds the suggested study sessions to the schedule when the student accepts.**

**The project will be analysed and designed with standard system analysis and design methods (requirements, use cases, UML class diagram, CRC cards) and built as a working prototype on the MERN stack.**

# 2. Problem Statement

**Students spread their school work across many tools, so nothing gives them one view of what is due and what they have actually studied. A typical student uses:**

- **the learning management system for assignment due dates**
- **a calendar or paper planner for study time**
- **a notes app or cloud folder for slides and readings**
- **memory, or nothing at all, for how much they have studied each course**

**Because these tools do not share information, a student can see that an exam is coming but not that they have studied that course much less than the others. They often notice the gap only a day or two before the exam. General-purpose planners store dates but do not compare deadlines with study effort, and they do not suggest what to do next.**

# 3. Objectives

**The project has five objectives, each with a way to check it at the end of the semester.**

| # | Objective | How I will check it |
|---|---|---|
| O1 | Give students one place for courses, tasks, deadlines, schedules and materials | Every feature in section 6 works in the prototype |
| O2 | Show what is due soon and warn before deadlines | Tasks due within 2 days appear as reminders on the dashboard |

| | | |
|---|---|---|
| **O3** | **Track study progress per course** | **Hours from finished sessions and logged time appear against each course's weekly goal** |
| **O4** | **Recommend what to study, based on deadlines and progress** | **For a test student with an exam in 3 days and 0 hours studied, the system recommends that course first** |
| **O5** | **Let the student act on a recommendation** | **Accepting a recommendation adds its sessions to the schedule; dismissing it hides it** |

# 4. Scope

**The project covers one student managing their own work through a web browser.**

**In scope**

- **Student accounts with sign-up and sign-in**
- **Courses, tasks (assignments, exams, quizzes, projects, readings), deadlines and priorities**
- **Study schedule with sessions, and logging of study time**
- **Upload and organisation of course files and links**
- **Progress tracking against a weekly goal per course**
- **In-app reminders for deadlines**
- **AI study recommendations that the student can accept or dismiss**
- **Updating and deleting tasks and study plans**

**Out of scope**

- **Importing data from a learning management system such as Blackboard or Canvas**
- **Instructor, parent or administrator roles**
- **Sharing or group study between students**
- **Native mobile apps (the web app will work in a phone browser)**
- **Grading, or generating quiz content from course materials**

# 5. Stakeholders and Users

**The student is the only user of the system; the other stakeholders shape or evaluate it.**

| Stakeholder | Role | Main interest |
|---|---|---|

| | | |
|---|---|---|
| **Student** | **Primary user** | **Keep track of work, meet deadlines, know what to study next** |
| **Course instructor** | **Evaluator of this project** | **A sound analysis and design, and a working prototype** |
| **Developer (me)** | **Analyst, designer and developer** | **A clear scope that can be finished in one semester** |
| **LLM provider (optional)** | **External system** | **Receives short requests to write recommendation text** |

# 6. Functional Requirements

**The system shall provide the following functions. Each one maps to a feature in the project README.**

| ID | Requirement | Feature |
|---|---|---|
| **FR1** | **The student can create an account, sign in and sign out** | **Accounts** |
| **FR2** | **The student can add, edit and delete courses with a code, name, colour and weekly study goal** | **Manage courses** |
| **FR3** | **The student can add assignments, exams, quizzes, projects and readings to a course** | **Manage tasks** |
| **FR4** | **Each task has a due date, a priority (high, medium, low) and an estimate of study hours needed** | **Deadlines and priorities** |
| **FR5** | **The system lists open tasks due in the next 14 days, soonest first** | **Upcoming deadlines** |
| **FR6** | **The student can add study sessions with a date, start time, length and focus note, and view them by week** | **Study schedule** |
| **FR7** | **The student can upload files (up to 20 MB each) and add links, organised by course** | **Course materials** |
| **FR8** | **The student can mark sessions done and log extra study time; the system shows hours studied against each weekly goal** | **Progress tracking** |
| **FR9** | **The system shows a reminder for every open task due within 2 days** | **Reminders** |

| | | |
|---|---|---|
| **FR10** | **The system recommends up to 3 courses to focus on, each with a reason and a suggested session plan** | **AI recommendations** |
| **FR11** | **The student can accept a recommendation, which adds its sessions to the schedule, or dismiss it** | **AI recommendations** |
| **FR12** | **The student can mark tasks done, edit them, and delete tasks and sessions** | **Update or delete** |

# 7. Non-Functional Requirements

| ID | Category | Requirement |
|---|---|---|
| **NFR1** | **Security** | **Passwords are stored as bcrypt hashes; every API request after sign-in needs a valid login token** |
| **NFR2** | **Privacy** | **A student can see and change only their own data; the LLM receives only course codes, task titles and hour counts** |
| **NFR3** | **Reliability** | **Recommendations still appear when the LLM is slow or unavailable, using rule-based text** |
| **NFR4** | **Performance** | **Pages load in under 2 seconds for a student with 10 courses and 200 tasks** |
| **NFR5** | **Usability** | **A new student can add a course and a task without instructions; the layout works on a phone browser** |
| **NFR6** | **Portability** | **The whole system starts with one Docker Compose command** |
| **NFR7** | **Maintainability** | **The recommendation logic is a separate module with unit tests** |

# 8. AI Study Recommendations

**The recommender is a hybrid: fixed rules decide what to recommend and how many hours, and a pretrained large language model (LLM) writes the explanation the student reads. The LLM never sets the numbers, so an invented hour or date cannot reach the schedule.**

**For each open task due in the next 14 days, the system computes the study gap and a priority score:**

$$gap = hours\ needed - hours\ studied - hours\ planned$$

*score = gap × w(priority) × w(type) × (0.4 + 1 / max(1, days left))*

**Hours studied are the finished sessions and logged time for that course in the 14 days before the deadline. Hours planned are unfinished sessions before the deadline. Weights favour high priority (1.6) over low (0.6), and exams (1.3) over readings (0.7).**

1. **The rule engine keeps tasks with a gap of at least 0.75 hours and ranks them by score.**
2. **For the top 3, it builds a session plan that spreads the gap over the days before the deadline, at most 2 hours per session where possible.**
3. **The LLM receives these facts and returns a one-sentence message and a short reason as JSON. The server checks the reply; if it is missing or malformed, a rule-based template is used instead.**
4. **The student accepts the recommendation, which adds the sessions to the schedule, or dismisses it, which hides it until the gap changes.**

**Any LLM with an OpenAI-compatible API can be used, for example Llama 3.2 run locally with Ollama at no cost, or a hosted model.**

# 9. Architecture and Technology

**The system is a three-tier MERN web application packaged with Docker, with an optional LLM for the recommendation text. The browser reaches the server only through /api; the server alone reads the database, stores files and calls the LLM.**

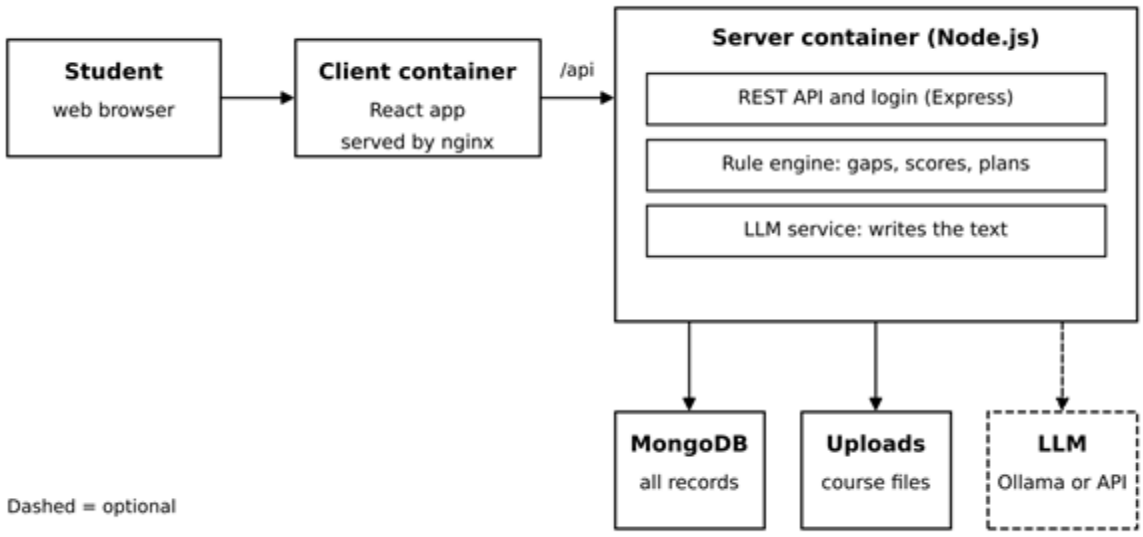



**Figure 1: System architecture**

| Layer | Technology | Purpose |
|---|---|---|
| Front end | React, Vite, React Router | Screens for dashboard, courses, tasks, schedule and materials |
| Back end | Node.js, Express | REST API, login tokens (JWT), file uploads (Multer) |
| Database | MongoDB with Mongoose | Users, courses, tasks, sessions, logs, materials and recommendations |
| AI | Rule engine + LLM (Ollama with Llama 3.2 by default) | Study recommendations |
| Deployment | Docker Compose, nginx | One command starts all containers |
| Tools | GitHub, Node test runner | Version control and unit tests |

# 10. Feasibility

**The project is feasible in one semester at no software cost.**

| Type | Assessment |
|---|---|
| Technical | All parts use mature, free, well-documented tools (MongoDB, Express, React, Node.js, Docker). A working prototype of the main screens and the recommendation rules already exists. |
| Operational | Students already keep deadlines and schedules; the system replaces several tools with one, and every recommendation can be ignored. |
| Economic | All software is open source. A local LLM through Ollama costs nothing; a hosted LLM is optional and only short text requests are sent. Hosting for the demo runs on my laptop. |
| Schedule | The scope in section 4 is limited to one user role and leaves out integrations, so it fits the 12-week plan in section 11. |

# 11. Schedule and Milestones

**The work runs over 12 weeks, with analysis and design complete before most of the coding starts.**

| Phase | Weeks | Milestone |
|---|---|---|

| | | |
|---|---|---|
| **Proposal and requirements** | **1 – 2** | **Proposal approved (end of week 2)** |
| **Analysis: use cases, UML, CRC cards** | **3 – 4** | |
| **Design: architecture and user interface** | **5 – 6** | **Design review (end of week 6)** |
| **Build core features** | **6 – 9** | |
| **Build AI recommendations** | **8 – 10** | **Working prototype (end of week 10)** |
| **Testing and fixes** | **10 – 11** | |
| **Final report and demo** | **12** | **Final report and demo (week 12)** |

**Week numbers count from the start of the project and can be matched to the course calendar.**

# 12. Risks and Mitigation

| Risk | Effect | Mitigation |
|---|---|---|
| **The LLM writes wrong or invented facts** | **Student is misled** | **The LLM only words the advice; numbers and plans come from the rule engine, and replies are validated** |
| **The LLM is slow, unavailable or costs money** | **Recommendations fail** | **Fall back to rule-based text; use a free local model through Ollama** |
| **Students do not enter study hours** | **Progress and advice are inaccurate** | **Tick-to-finish sessions and one-click time logging on the dashboard** |
| **Poor time estimates for tasks** | **Plans are too short or too long** | **Show the estimate in each recommendation so the student can correct it** |
| **Scope grows during the semester** | **Deadline missed** | **Fixed scope in section 4; new ideas go to a future-work list** |
| **Personal data exposed** | **Loss of privacy** | **Hashed passwords, per-user data access, no personal data sent to the LLM** |

# 13. Deliverables and Conclusion

**The project will deliver:**

- **This proposal**
- **Analysis documents: requirements, use case diagram and descriptions**
- **Design documents: UML class diagram, CRC cards, architecture diagram**
- **A working prototype in a public repository, started with Docker Compose**
- **Unit tests for the recommendation engine**
- **A final report and an in-class demonstration**

**The AI Study Assistant addresses a real gap: students can see their deadlines but not whether their study time matches them. By joining planning, progress tracking and explainable AI recommendations in one tool, the system helps students decide what to study next and act on that decision.**